Clover: AI-Powered Academic Writing

A Local-First Approach to Intelligent Paper Authoring

February 2026

Note: This document was written as a sample project to demonstrate Clover's capabilities, and was generated with the assistance of a large language model (Claude) via Clover. The content may contain inaccuracies or hallucinations inherent to LLM-generated text.

Abstract

Academic writing has long suffered from tool fragmentation—researchers juggle LaTeX editors, reference managers, AI assistants, and collaboration platforms as disconnected pieces of a broken workflow. Clover is a local-first, AI-integrated LaTeX writing environment that unifies these capabilities into a single desktop application. By embedding Claude Code directly alongside a LaTeX editor and a NotebookLM-style source manager, Clover enables researchers to write, cite, compile, and review within one coherent workspace—all while keeping their data under their own control. This book introduces Clover's design philosophy, walks through its core features, and explains why the local-first approach matters for the future of academic writing.

Contents

1 Introduction: The State of Academic Writing **3**
- 1.1 The Fragmented Landscape . . . 3
- 1.2 Cloud-Based LaTeX Editors . . . 3
- 1.3 Modern Typesetting Alternatives . . . 4
- 1.4 AI-Powered Research Tools . . . 4
- 1.5 Fully Automated Paper Generation . . . 5
- 1.6 General AI Assistants . . . 5
- 1.7 The Gap . . . 5

2 Clover's Design Philosophy **7**
- 2.1 Data Ownership and Vendor Independence . . . 7
- 2.2 Privacy for Sensitive Research . . . 7
- 2.3 Git as a First-Class Citizen . . . 8
- 2.4 Offline Capability and Resilience . . . 9
- 2.5 Extensibility Through Openness . . . 9
- 2.6 Local-First as Architectural Commitment . . . 10

3 What is Clover? **11**
- 3.1 A Unified Workspace . . . 11
- 3.2 Technical Foundation . . . 12
- 3.3 Claude Code Integration . . . 12
- 3.4 How the Pieces Connect . . . 13
- 3.5 More Than the Sum of Its Parts . . . 14

4 The Integrated LaTeX Editor **15**
4.1 Monaco Editor for LaTeX . . . 15
4.2 AI-Assisted Writing Through Claude Code . . . 16
4.3 Git Diff View for Transparency . . . 16
4.4 PDF Compilation and Preview . . . 17
4.5 A Workspace, Not a Collection of Tools . . . 18

5 Source Management: Your Research Hub **19**
5.1 Adding and Registering Sources . . . 19
5.2 The Tag System: Structured Discovery . . . 19
5.3 AI-Powered Source Utilization . . . 20
5.4 Beyond NotebookLM: Sources as a Writing Substrate . . . 21
5.5 A Living Knowledge Base . . . 21

6 Clover Skills: Smart Writing Assistance **23**
6.1 Brainstorming: From Vague Ideas to Concrete Plans . . . 23
6.2 Paper Research: Assisted Literature Discovery . . . 24
6.3 Source Registration: Streamlined Reference Management . . . 24
6.4 Translation: Academic-Quality Multilingual Adaptation . . . 25
6.5 The Philosophy Behind Skills . . . 26

7 Getting Started & Future Directions **27**
7.1 Prerequisites . . . 27
7.2 Creating a Project . . . 27
7.3 The Basic Writing Workflow . . . 28
7.4 Future Directions . . . 28
7.5 Conclusion . . . 29

Chapter 1: Introduction: The State of Academic Writing

Since the release of ChatGPT in late 2022, a wave of artificial intelligence tools has swept through every corner of knowledge work—and academic writing has been no exception. By 2024, researchers at every career stage had begun experimenting with large language models for drafting, editing, summarizing, and even generating entire manuscripts. The pace of change has only accelerated: 2025 saw the launch of specialized AI research agents, and in January 2026 OpenAI entered the arena directly with Prism [1], a free LaTeX-native workspace for scientists powered by GPT-5.2. The message is unmistakable. AI is no longer a curiosity at the margins of scholarly communication; it is rapidly becoming infrastructure. Yet for all this activity, the daily experience of writing a research paper remains remarkably fragmented, and the tools that promise to help often introduce as many frictions as they resolve.

1.1 The Fragmented Landscape

Consider the workflow of a typical researcher preparing a journal submission in early 2026. She drafts her manuscript in Overleaf [2] or a local LaTeX editor such as TeXShop or VS Code. Her references live in Zotero [3] or Mendeley [4], each running as a separate desktop application with its own database, its own sync service, and its own idiosyncratic export pipeline for producing `.bib` files. When she wants AI assistance—to rephrase an awkward paragraph, to brainstorm a stronger framing for her introduction, or to check whether her argument follows logically—she opens a browser tab to ChatGPT or Claude, copies and pastes a block of text, waits for a response, and then manually transfers the result back into her editor. If she needs to verify a claim against the literature, she might turn to Google's NotebookLM [5] for source-grounded analysis, or to Elicit [6] for systematic evidence search across its index of over 138 million papers. Each of these tools performs its designated task competently, sometimes brilliantly. But none of them talks to the others. The researcher is the integration layer, shuttling text and citations between applications, losing context at every boundary, and spending cognitive effort on logistics that should be invisible. This constant context-switching is not merely inconvenient; it actively degrades the quality of scholarly work by fragmenting the writer's attention at precisely the moments when sustained, deep thought matters most.

1.2 Cloud-Based LaTeX Editors

Among existing tools, Overleaf [2] occupies the dominant position in collaborative academic typesetting. With over twenty million registered users, it has become the de facto standard for multi-author LaTeX projects, particularly in the sciences and engineering. Its strengths are genuine: real-time collaboration without the pain of merge conflicts, a browser-based interface that eliminates local installation headaches, over a thousand journal templates, and a visual editor mode that lowers the barrier for LaTeX newcomers. In 2025, its parent company Digital Science introduced AI Assist, an add-on powered by Writefull that provides grammar checking, LaTeX error assistance, code generation from natural-language prompts, and rewriting tools for paraphrasing and tone adjustment.

Yet Overleaf's architecture carries fundamental trade-offs that no amount of incremental improvement can fully resolve. As a cloud-first platform, it requires a persistent internet connection for every keystroke. Compilation runs on remote servers, which means that during periods of heavy load—conference deadlines being the most notorious example—authors face queuing delays that can stretch from seconds to minutes. The free tier imposes a ten-second compile timeout and limits collaboration to a single co-author, pushing serious users toward paid subscriptions that, for institutional teams, can reach several hundred dollars per year. More subtly, Overleaf creates vendor lock-in: while projects can be exported as ZIP archives, the collaborative history, comments, and review annotations are trapped inside the platform. Researchers who

depend on Overleaf are, in effect, renting their writing environment rather than owning it. And despite the AI Assist add-on, Overleaf's AI capabilities remain shallow—limited to sentence-level corrections and short code snippets, with no understanding of the project's broader structure, argumentation, or source material.

1.3 Modern Typesetting Alternatives

Typst [7] has emerged as the most promising modern alternative to LaTeX itself. Written in Rust, it compiles documents in milliseconds rather than seconds, uses a cleaner markup syntax inspired by Markdown, and offers native Unicode support, incremental compilation, and a web-based collaborative editor. For newcomers to academic typesetting, Typst's learning curve is far gentler than LaTeX's notoriously arcane syntax, and its error messages are substantially more informative.

However, Typst remains a young project with an ecosystem that has not yet reached critical mass. Its package repository, Typst Universe, contains a fraction of the templates and packages available in the CTAN archive that LaTeX has accumulated over four decades. Journal submission pipelines overwhelmingly expect `.tex` files; very few publishers accept Typst source directly. The community, while enthusiastic, is small, which means that troubleshooting obscure formatting requirements often leads to dead ends. Typst also does not address the broader workflow fragmentation described above—it is a better typesetting engine, not an integrated writing environment. A researcher who switches from LaTeX to Typst still faces the same disconnected constellation of reference managers, AI assistants, and source-analysis tools.

1.4 AI-Powered Research Tools

A parallel category of tools has emerged to apply AI specifically to the research process rather than to typesetting. Google's NotebookLM [5] exemplifies this approach: users upload sources—PDFs, web pages, Google Docs—and the system, powered by Gemini, provides source-grounded answers that resist hallucination by anchoring every response to the uploaded material. Its Deep Research feature can autonomously browse the web and synthesize fifteen to twenty-five sources into a structured research report in under five minutes. These capabilities are genuinely powerful for literature comprehension and exploratory analysis.

Elicit [6] and SciSpace [8] occupy a related niche, focusing on literature discovery and evidence synthesis. Elicit searches across 138 million papers and 545,000 clinical trials, offering systematic review workflows with data extraction and sentence-level citations for transparency. SciSpace integrates over 150 tools and databases, providing automated mini literature reviews, AI-assisted manuscript drafting, and even journal-matching recommendations. Both platforms have introduced agent-based workflows that can execute multi-step research tasks with minimal human intervention.

OpenAI's Prism [1], launched in January 2026, represents the most ambitious entry from a major AI lab. It combines a LaTeX-native document editor with GPT-5.2 for idea exploration, hypothesis testing, and step-by-step reasoning, alongside citation management and collaborative features—all offered for free to ChatGPT account holders. Prism is significant because it acknowledges, for the first time from a frontier-model provider, that scientific writing deserves a dedicated workspace rather than a generic chat interface.

Yet each of these tools addresses only a subset of the writing workflow. NotebookLM is a source-analysis environment, not a writing tool: it has no LaTeX editor, no compilation pipeline, and no way to produce a formatted manuscript. Elicit and SciSpace excel at literature search but do not support document authoring. Prism comes closest to an integrated solution but is entirely cloud-dependent, closed-source, and tethered to a single AI provider—researchers who use it surrender both their data and their choice of model to OpenAI's infrastructure. None of

these platforms allow the writer to work offline, to choose their own AI backend, or to maintain full ownership of their project files on their own machine.

1.5 Fully Automated Paper Generation

At the far end of the automation spectrum, a growing body of work seeks to remove the human author from the loop entirely. The AI Scientist [9], introduced by Sakana AI in 2024, demonstrated that a multi-agent system could autonomously generate research ideas, implement code, run experiments, write complete papers in LaTeX, and even conduct simulated peer review—all for less than fifteen dollars per paper. Agent Laboratory [10] extended this paradigm by introducing a three-phase pipeline (literature review, experimentation, report writing) with specialized LLM agents collaborating via arXiv, Hugging Face, Python, and LaTeX, allowing optional human feedback at each stage.

These systems are intellectually fascinating and technically impressive, but they raise profound questions about quality, verification, and scientific responsibility. Independent evaluations of AI Scientist have produced mixed results, with generated papers sometimes exhibiting logical gaps, fabricated references, or superficial experimental analysis that would not survive rigorous peer review. More fundamentally, the philosophy of full automation misunderstands what most researchers actually need. The bottleneck in academic writing is rarely the mechanical act of producing text; it is the conceptual work of formulating arguments, interpreting evidence, and making judgment calls about what to include, what to emphasize, and what to leave out. Systems that automate the entire pipeline do not solve this problem—they bypass it, and in doing so they remove precisely the human judgment that gives scholarship its epistemic value. The goal should not be to replace the researcher but to amplify her capabilities.

1.6 General AI Assistants

For many researchers today, the most commonly used AI writing tool is simply a general-purpose assistant: ChatGPT or Claude accessed through a web browser or API [11]. These models are remarkably capable at prose generation, style editing, logical analysis, and even LaTeX code generation when prompted carefully. A researcher can paste a paragraph and receive a tightened rewrite in seconds, or describe a table structure in natural language and get back compilable LaTeX.

The limitations, however, are structural rather than intellectual. A general AI assistant has no persistent awareness of the researcher's project. Every conversation starts from zero: the model does not know which papers have been cited, what the document's overall structure looks like, which sections are finished and which are still in draft, or what style conventions the target journal requires. The workflow is fundamentally copy-and-paste: the researcher extracts a fragment from her editor, switches to the chat interface, provides context that the model should already have, receives a response, evaluates it, and manually transfers the useful portions back into her document. This process is tedious, error-prone, and scales poorly. It also introduces subtle risks: text generated outside the editor can introduce inconsistencies in notation, terminology, or citation style that are difficult to catch during manual integration.

1.7 The Gap

What emerges from this survey is a clear structural absence in the landscape of academic writing tools. On one side stand powerful but narrow instruments—reference managers that know nothing about your prose, AI assistants that know nothing about your sources, typesetting engines that know nothing about your intent. On the other side stand fully automated systems that know everything about your project but leave no room for your judgment. Between these extremes lies an unoccupied territory: a tool that combines LaTeX editing, source management, and AI

assistance in a single, local-first environment, one that keeps the human author firmly in control while removing the friction that currently makes academic writing so needlessly laborious.

This is the space that Clover is designed to fill. In the chapters that follow, we will examine the design philosophy behind Clover's architecture (Chapter 2), introduce its core capabilities (Chapter 3), and walk through its integrated LaTeX editor (Chapter 4), source management system (Chapter 5), and intelligent writing skills (Chapter 6), before discussing how to get started and where the project is headed (Chapter 7). The aim is not to replace any single tool in the researcher's existing arsenal but to provide a coherent environment in which those capabilities—editing, citation, compilation, and AI-powered assistance—finally work together as a unified whole.

Chapter 2: Clover's Design Philosophy

Every design decision begins with a philosophical commitment, and Clover's is stated plainly: your files, your machine, your control. In a landscape increasingly dominated by cloud platforms that treat the researcher's manuscript as a tenant on someone else's server, Clover takes the opposite stance. It is built from the ground up as a local-first application [12], meaning that the primary copy of every document, every source, and every configuration file lives on the researcher's own filesystem. The cloud is available when useful—for AI assistance, for collaboration, for backup—but it is never required and never authoritative. This chapter explains why local-first is not merely a technical preference but a deliberate architectural choice with far-reaching implications for data ownership, privacy, resilience, and the long-term autonomy of the researchers who depend on their tools.

2.1 Data Ownership and Vendor Independence

When a researcher writes in Clover, every file is a plain LaTeX document stored in an ordinary directory on the local filesystem. There is no proprietary format, no opaque database, no binary blob that can only be opened with the application that created it. A `.tex` file written in Clover is identical to a `.tex` file written in Emacs, VS Code, TeXShop, or any other editor—because it *is* the same file. The bibliography is standard BibTeX. The images are standard PNG or PDF. The project structure is a directory tree that any operating system can navigate, any backup tool can archive, and any version control system can track.

This simplicity has profound consequences for vendor independence. A researcher who uses Overleaf [2] can, in principle, export her project as a ZIP archive, but the export is an afterthought: collaborative history, inline comments, review annotations, and track-changes metadata are all trapped inside the platform. The project's authoritative copy lives on Overleaf's servers, and the researcher's relationship to her own manuscript is mediated entirely by the platform's interface, pricing tiers, and continued existence. If Overleaf changes its terms of service, raises its subscription fees, or suffers a prolonged outage, the researcher is exposed—not because her LaTeX source is technically inaccessible, but because her workflow, her history, and her collaborative context are locked in. The same structural dependency applies to OpenAI's Prism [1], which offers a free and powerful writing environment but requires that every document, every revision, and every AI interaction pass through OpenAI's cloud infrastructure.

Clover rejects this model entirely. Because the filesystem is the single source of truth, switching editors is trivial—open the same directory in a different application and continue working. Moving to a new machine means copying a folder. Archiving a project for long-term preservation means storing a directory alongside the TeX Live distribution that compiles it. Kleppmann et al. describe this property as "the long now": the principle that software should not hold your data hostage and that documents should remain readable for decades, not merely for as long as a particular company remains in business [12]. Clover honors this principle by adding nothing to the file format. The application creates value through its integrated workspace—the editor, the source manager, the AI assistant—but the output is always plain, portable, standard LaTeX.

2.2 Privacy for Sensitive Research

Researchers routinely handle material that is sensitive, confidential, or simply not yet ready for public exposure. Unpublished manuscripts represent months or years of intellectual effort and competitive advantage. Proprietary datasets obtained under non-disclosure agreements carry legal obligations. Grant proposals contain strategic plans that would be compromised by premature disclosure. Peer review assignments are confidential by professional norm and often by contractual requirement. Patient data, interview transcripts, and classified research materials carry even stricter regulatory constraints.

Cloud-based writing tools require the researcher to trust a third party with all of this content. Every keystroke in Overleaf travels to Digital Science's servers. Every document uploaded to Prism passes through OpenAI's infrastructure. Every source added to NotebookLM [5] enters Google's cloud. For many researchers, particularly those working with human subjects data, export-controlled technology, or corporate-sponsored research, this trust cannot be granted—not because the platforms are necessarily malicious, but because institutional review boards, funding agencies, and legal departments often prohibit storing sensitive data on external servers that fall outside the institution's direct control.

Clover's local-first architecture addresses this concern at the structural level rather than through policy promises. Source files never leave the researcher's filesystem. The project directory can reside on an encrypted volume, behind institutional firewalls, or on an air-gapped machine, and Clover will function identically in each case. The one component that does communicate with an external service is the AI assistant: when the researcher invokes Claude Code [11] for writing assistance, the relevant content is sent to Anthropic's API for processing, subject to Anthropic's data handling and privacy commitments. But this communication is explicit, user-initiated, and limited to the specific content the researcher chooses to share. The rest of the project—the source files, the bibliography, the compiled output, the project configuration—remains entirely local. This separation of concerns means that a researcher can use Clover's editing, compilation, and source management features with complete confidence that no data is transmitted anywhere, and can then choose, deliberately and selectively, when to engage the AI component and what content to expose to it.

2.3 Git as a First-Class Citizen

Version control is essential to any serious writing project, yet most academic writing tools either ignore it or abstract it away into a proprietary layer. Overleaf offers a "History" feature that records snapshots at system-determined intervals, and premium users can access a Git integration that allows cloning projects as repositories. However, the exported Git history reflects sync points rather than the full granular edit timeline, and the integration is unavailable on the free tier. Prism provides revision tracking within its cloud environment, but the revisions are not Git commits and cannot be inspected with standard tools. In both cases, the researcher is presented with a simplified view of the document's evolution rather than a full, branching, annotatable history under her own control.

Clover embraces Git natively. Because every project is a directory of plain files, it is already a valid Git repository—or can become one with a single command. Every edit the researcher makes is visible in the working tree. Every AI-generated change proposed by Claude Code appears as a modification that can be inspected in a standard diff view, accepted or rejected line by line, and committed with a descriptive message. The researcher can create branches to experiment with alternative structures, merge contributions from collaborators, and maintain a complete, cryptographically signed history of the document's evolution from first outline to final submission.

This transparency is especially important in the context of AI-assisted writing. When a language model rewrites a paragraph, the change is not hidden behind a "magic" button that silently replaces text. It appears in the diff, attributed and reviewable, just like any other edit. The researcher can see exactly what the AI proposed, compare it with the original, and make an informed decision about whether to accept, modify, or discard the suggestion. This workflow aligns with the growing expectation in academic publishing that authors take responsibility for every word in their manuscript, including words drafted with AI assistance. A Git history that records each AI contribution as a discrete, inspectable commit provides an audit trail that no cloud-based "track changes" feature can match.

For collaborative projects, Git's distributed architecture is a natural complement to Clover's local-first design. Each collaborator works with a full copy of the repository on their own machine.

Synchronization happens through push and pull operations to a shared remote—GitHub, GitLab, a departmental server, or any other Git hosting provider. There is no single point of failure, no platform lock-in, and no dependency on a proprietary collaboration layer. Researchers who already use Git for code can use the same tools, the same workflows, and the same mental model for their writing projects.

2.4 Offline Capability and Resilience

A local-first tool works without the internet. This is not a theoretical edge case; it is a practical reality that researchers encounter regularly. Writing sessions happen on airplanes, in libraries with unreliable Wi-Fi, in hotel rooms during conferences, in field stations with satellite connections measured in kilobits per second, and in countries where access to particular cloud services is restricted or monitored. A tool that requires a persistent network connection to function—as Overleaf does for every keystroke and compilation—is a tool that abandons its users precisely when uninterrupted writing time is most available and most valuable.

Clover's core functionality operates entirely offline. The editor renders and responds to input locally. The LaTeX compiler runs on the researcher's machine using a locally installed TeX distribution. The source manager reads and indexes files from the local filesystem. The PDF previewer displays output generated locally. A researcher can open Clover on a disconnected laptop, write for hours, compile repeatedly, reorganize sources, and produce a finished PDF without ever transmitting a single byte over the network.

The AI-powered features—Claude Code's writing assistance, deep research, and source-grounded analysis—do require an internet connection, because they depend on Anthropic's API. But this dependency is compartmentalized: when the network is unavailable, the AI features are simply inactive, and every other aspect of the application continues to function normally. There is no spinner, no error dialog, no degraded mode that disables unrelated features as a side effect of network loss. This graceful degradation is a direct consequence of the local-first architecture: because the application does not depend on a server for its core operations, the absence of a server is a loss of one capability, not a collapse of the entire system. Kleppmann et al. identify this property—"the network is optional"—as one of the seven ideals of local-first software, and Clover implements it faithfully [12].

Resilience extends beyond intermittent connectivity. Cloud services experience outages, sometimes at the worst possible moments. Overleaf's compilation servers have historically slowed to a crawl during major conference deadlines, precisely when researchers most need reliable performance. A local-first tool is immune to this failure mode: performance depends on the researcher's own hardware, which does not degrade because thousands of other users happen to be compiling at the same time.

2.5 Extensibility Through Openness

Because Clover works with standard files and standard tools, it does not ask the researcher to abandon her existing workflow. It adds to the toolchain rather than replacing it. A researcher who manages references with Zotero [3] can continue to do so: Zotero exports BibTeX, and Clover reads BibTeX. A researcher who prefers a particular Git client—whether that is the command line, GitHub Desktop, Tower, or Magit inside Emacs—can use it alongside Clover, because the underlying repository is a standard Git repository with no proprietary metadata. A researcher who has invested years in customizing a LaTeX preamble, developing personal style files, or assembling a collection of TikZ macros can bring all of that into Clover without modification, because Clover compiles LaTeX the same way any other tool does: by invoking the local TeX distribution.

This openness contrasts sharply with the walled-garden approach of cloud platforms. Overleaf supports a curated set of LaTeX packages tied to the TeX Live version installed on its servers; if

a researcher needs a package that is not included, she must wait for Overleaf to update or resort to uploading the package files manually. Prism's AI features are tethered to GPT-5.2; there is no option to use a different model, even if a specialized or fine-tuned model would better serve the researcher's domain. In both cases, extensibility is constrained by the platform's choices rather than the researcher's needs.

Clover's architecture inverts this relationship. The researcher controls the TeX distribution, the packages, the bibliography tools, the Git workflow, and—through Claude Code's support for the Model Context Protocol and custom commands—even the behavior of the AI assistant itself. Clover provides a coherent, integrated workspace, but it does so by orchestrating standard components rather than by replacing them with proprietary equivalents. The result is a tool that respects the researcher's existing investments and adapts to her workflow, rather than demanding that she adapt to it.

2.6 Local-First as Architectural Commitment

Local-first is not a limitation, and it is not a nostalgic retreat from the cloud. It is a deliberate architectural commitment that prioritizes researcher autonomy over platform convenience, long-term portability over short-term ease of onboarding, and transparent, inspectable workflows over opaque, server-mediated ones. The seven ideals articulated by Kleppmann et al.—speed, multi-device access, offline operation, seamless collaboration, longevity, privacy, and user control [12]—are not abstract principles for Clover; they are concrete design constraints that shape every feature described in the chapters that follow.

Clover demonstrates that an AI-powered writing tool can be both powerful and respectful of its users' data and workflow. The AI does not require surrendering your files to a cloud service. Version control does not require trusting a proprietary platform with your history. Compilation does not require waiting in a server queue. Source management does not require uploading your research to a third party's infrastructure. Each of these capabilities works locally, under the researcher's direct control, with the cloud serving as an optional enhancement rather than a mandatory dependency. In an era when academic writing tools are converging toward centralized, cloud-dependent, vendor-locked platforms, Clover offers a different path—one where the researcher remains the owner of her tools, her data, and her intellectual work.

Chapter 3: What is Clover?

Clover is a desktop application that combines a LaTeX editor, a live PDF preview, an AI terminal running Claude Code, and a source manager into one unified workspace for academic writing. Where existing tools force researchers to scatter their attention across disconnected applications—an editor here, a reference manager there, a chat window with an AI assistant in another corner of the screen—Clover consolidates the entire writing workflow into a single window. The result is an environment in which the distance between thinking and writing, between reading a source and citing it, between asking an AI for help and seeing the result in your manuscript, collapses to zero.

3.1 A Unified Workspace

The Clover interface is organized around a set of panels that can be arranged, resized, and toggled according to the researcher's preferences, but whose default layout reflects the application's core philosophy: everything the writer needs should be immediately accessible without switching windows or applications.

The editor panel occupies the central position. It provides a full-featured LaTeX editing environment with syntax highlighting, bracket matching, code folding, autocompletion, and all the standard capabilities that researchers expect from a modern text editor. This is where the manuscript lives and where the writer spends most of her time. The editor understands LaTeX at a structural level, not merely as plain text, which means it can offer intelligent completions for commands, environments, and citation keys drawn from the project's bibliography.

Adjacent to the editor sits the PDF preview panel, which displays the compiled output of the manuscript. When the researcher triggers a compilation—or when the system compiles automatically—the preview refreshes to show the current state of the document as it will appear in print. This immediate visual feedback closes the loop between authoring and typesetting: the writer does not need to leave the application, open a separate PDF viewer, or mentally map source lines to rendered pages. The preview and the source are side by side, and the correspondence between them is always visible.

The terminal panel runs Claude Code [11], Anthropic's AI coding and writing assistant, directly inside the Clover workspace. This is not a small chat widget or a sidebar offering sentence-level suggestions. It is a fully functional terminal emulator that provides access to a powerful AI agent capable of reading the entire project, understanding the document's structure and argumentation, writing new sections, editing existing content, managing references, and executing multi-step tasks that span dozens of files. The terminal panel makes the AI a first-class participant in the writing process rather than an external tool consulted through a browser tab.

The source management panel provides a structured interface for organizing the research materials that inform the manuscript. Sources—PDFs, web pages, notes, data files—are collected, tagged, and made available to both the researcher and the AI assistant. This panel is inspired by the source-grounded approach pioneered by Google's NotebookLM [5], adapted to the specific needs of LaTeX-based academic writing. When a source is registered in the manager, it becomes part of the project's knowledge base: the AI can reference it, the researcher can search it, and citations can be generated from it.

Finally, the Git diff view provides a panel for reviewing changes to the manuscript. Because Clover projects are standard Git repositories, every edit—whether made by the researcher or by the AI—is tracked as a modification to plain text files. The diff view shows exactly what has changed, line by line, making it straightforward to review AI-generated contributions with the same rigor one would apply to a pull request in a software project.

The power of this layout is not in any single panel but in their co-presence. Having the editor, the preview, the AI terminal, the source manager, and the diff view all visible within

one window eliminates the context-switching that plagues fragmented workflows. The researcher does not lose her train of thought navigating between applications. She does not copy and paste text between an editor and a chat interface. She does not manually transfer a citation from a reference manager into her `.bib` file. The panels communicate through the shared filesystem and the shared project state, creating an integrated experience that is qualitatively different from a collection of standalone tools arranged on the same screen.

3.2 Technical Foundation

Clover is built as an Electron desktop application, a choice that provides cross-platform compatibility across macOS, Windows, and Linux while allowing the application to leverage the full capabilities of the local operating system—file system access, process management, and native compilation toolchains. For researchers who prefer not to install a desktop application, or who need to access their workspace from a machine that does not have Clover installed, the same interface is also available as a web application served by an Express-based web server. The desktop and web versions share the same codebase, ensuring feature parity regardless of how the researcher chooses to access the tool.

The frontend is built with React and TypeScript, providing a component-based architecture that makes the interface responsive and maintainable. The LaTeX editor is powered by Monaco Editor, the same editing engine that underlies Visual Studio Code—arguably the most widely used code editor in the world. By adopting Monaco, Clover inherits a mature, battle-tested editing experience: fast rendering even for large documents, sophisticated syntax highlighting, multi-cursor editing, configurable keybindings, and an extensive API for editor extensions. Researchers who are already familiar with VS Code will find the editing experience immediately recognizable.

Terminal emulation is handled by xterm.js, the same library used by VS Code's integrated terminal, by Hyper, and by numerous other modern terminal applications. This ensures that the Claude Code session running inside Clover behaves exactly as it would in a standalone terminal: full ANSI color support, proper cursor handling, scrollback history, and reliable input processing. The terminal is not a simplified text box that approximates a command-line interface; it is a complete terminal emulator that can run any command-line tool the researcher needs, with Claude Code as its primary occupant.

PDF rendering is provided by react-pdf, which displays compiled documents directly within the application window. The interface is styled with Tailwind CSS, a utility-first CSS framework that enables rapid, consistent visual design. Together, these technologies produce an application that feels native and responsive despite being built on web technologies—a testament to how far the Electron ecosystem has matured since its early days of sluggish, memory-hungry applications.

3.3 Claude Code Integration

The terminal panel in Clover runs Claude Code [11], and this integration is the single most consequential design decision in the application. Claude Code is not a chatbot grafted onto a text editor. It is an agentic AI assistant that operates directly on the project's files with the same level of access that the researcher herself possesses. It can read every `.tex` file in the project directory, parse the document's sectional structure, examine the bibliography, inspect source materials registered in the source manager, and write or modify any file in the project tree.

When the researcher asks Claude Code to draft a new section, the AI reads the existing manuscript to understand its tone, structure, and argument, then writes LaTeX directly into the appropriate file. When she asks it to add a citation, it consults the bibliography, identifies the correct key, and inserts a `\cite{}` command in the right place. When she asks it to reorganize a chapter, it can move content between files, update cross-references, and adjust the document's

logical flow—all within the same terminal session, all without the researcher needing to copy, paste, or manually integrate anything.

This depth of integration distinguishes Clover from the shallow AI features found in most writing tools. Overleaf's AI Assist [2] offers grammar corrections and short code snippets; it does not understand the document as a whole. General-purpose assistants like ChatGPT or Claude accessed through a browser have no persistent access to the project; every interaction starts from a blank slate. Even OpenAI's Prism [1], which embeds AI more deeply than most competitors, mediates the interaction through a cloud interface that the researcher does not control. In Clover, Claude Code runs locally in the researcher's terminal, reads files from the local filesystem, and writes changes that appear immediately in the editor and the Git working tree. The AI is not a service the researcher visits; it is a collaborator that shares her workspace.

Claude Code also supports the Model Context Protocol (MCP) [11], which allows it to connect to external tools and data sources—web search, database queries, API calls—extending its capabilities beyond the local filesystem when the researcher's task demands it. Custom commands and persistent instructions via a `CLAUDE.md` configuration file allow the researcher to tailor the AI's behavior to the specific conventions of her project, her discipline, or her target journal. The result is not a one-size-fits-all assistant but an adaptable agent whose behavior can be shaped to match the researcher's exact needs.

3.4 How the Pieces Connect

The individual components of Clover—the editor, the preview, the terminal, the source manager, the diff view—are powerful in isolation, but what makes the application distinctive is the way they interact through the shared project state.

When Claude Code writes LaTeX in the terminal, the editor panel updates to reflect the new content. The researcher can see the AI's output appear in real time, review it in the editor's syntax-highlighted view, and make immediate adjustments if needed. When she triggers a compilation, the PDF preview refreshes to show the rendered result, closing the loop from AI-generated source code to typeset output in a matter of seconds. If she is unsatisfied with a passage, she can ask Claude Code to revise it, watch the editor update, recompile, and evaluate the new rendering—all without leaving the window.

When the researcher adds a new source to the source manager—uploading a PDF, registering a URL, or importing a BibTeX entry—that source becomes immediately available to Claude Code in the next writing session. The AI can read the source, extract relevant claims, and incorporate them into the manuscript with proper citations. This eliminates the laborious manual pipeline of reading a paper in one application, noting a key finding, switching to the editor, looking up the citation key, and typing the reference by hand. In Clover, the path from source to citation is direct and AI-assisted.

Every change, whether made by the researcher's hand or by the AI's output, is tracked by Git. The diff view shows precisely what was added, modified, or removed, and every modification can be committed with a descriptive message that records the intent behind the change. This creates an audit trail that is especially valuable for AI-assisted writing: a reviewer or collaborator can inspect the Git history to see which passages were drafted by the AI and which were written by the researcher, providing the transparency that responsible use of AI in academic work demands.

This tight integration means that the AI is not an external tool the researcher consults when she gets stuck. It is part of the writing environment itself, operating on the same files, reading the same sources, and producing output that flows directly into the same compilation pipeline. The boundary between human writing and AI-assisted writing dissolves—not because the distinction does not matter, but because the tools no longer force an artificial separation between the two modes of work.

3.5 More Than the Sum of Its Parts

Clover is not merely a collection of features arranged in a single window. LaTeX editors exist. PDF previewers exist. AI assistants exist. Source managers exist. Git clients exist. A researcher could, in principle, open five applications side by side and approximate the Clover experience through sheer force of window management. But the friction of that arrangement—the constant switching, the manual data transfer, the loss of context at every application boundary—is precisely the problem that Clover is designed to solve. The value of Clover lies not in inventing new capabilities but in the opinionated integration of existing ones: the decision that these tools belong together, that they should share a common project state, and that the AI should have the same access to the manuscript that the human author does. It is this integration, more than any single feature, that creates a writing experience greater than the sum of its parts. The chapters that follow will examine each component in detail, beginning with the LaTeX editor in Chapter 4 and the source management system in Chapter 5, before exploring the intelligent writing skills that tie everything together in Chapter 6.

Chapter 4: The Integrated LaTeX Editor

The LaTeX editor is where a researcher spends the vast majority of her writing time. Every other feature in Clover—the source manager, the AI assistant, the PDF preview, the Git integration—exists ultimately to serve what happens in the editor panel. If the editing experience is clumsy, slow, or impoverished, no amount of surrounding intelligence can compensate. Clover's editor is therefore designed from the ground up to make that time as productive as possible, combining a professional-grade text editor with AI-powered writing assistance, transparent change tracking, and immediate compilation feedback. This chapter examines each of these capabilities in detail, showing how they work individually and how their integration within a single interface creates a writing experience that is qualitatively different from the sum of its parts.

4.1 Monaco Editor for LaTeX

The foundation of Clover's editing experience is the Monaco Editor, the same editing engine that powers Visual Studio Code. By building on Monaco rather than creating a bespoke editor from scratch, Clover inherits decades of accumulated engineering investment in text editing performance, accessibility, and extensibility. The practical consequence for the researcher is an editor that feels immediately familiar if she has ever used VS Code, and that meets the expectations of anyone accustomed to a modern code editor even if she has not.

Monaco provides syntax highlighting for LaTeX that makes the structure of a document visible at a glance. Commands appear in one color, arguments in another, math environments are distinguished from text environments, and comments are dimmed so that they do not compete for visual attention with the manuscript's actual content. This chromatic structure is not merely aesthetic; it serves as a real-time error-detection mechanism. A missing closing brace, an unclosed environment, or a mistyped command name disrupts the expected color pattern, alerting the writer to the problem before she ever attempts to compile. For researchers who work with complex LaTeX documents—nested environments, custom macros, multi-file projects with dozens of `\input{}` and `\include{}` directives—this immediate visual feedback is invaluable.

Beyond syntax highlighting, Monaco offers bracket matching that highlights corresponding delimiters when the cursor rests on one of them, code folding that allows the writer to collapse sections, environments, or lengthy preambles to focus on the passage currently under revision, and multi-cursor editing that enables simultaneous changes at multiple points in the document. Auto-completion suggests LaTeX commands, environment names, and—crucially—citation keys drawn from the project's `.bib` file, reducing the friction of inserting references from a manual lookup to a few keystrokes. Configurable keybindings allow researchers to bring their muscle memory from other editors, whether that means Vim-style modal editing, Emacs keychords, or the default bindings that VS Code users already know by heart.

The choice of Monaco also has architectural implications that benefit Clover's long-term development. Because Monaco is actively maintained by Microsoft's VS Code team and supported by a large open-source community, improvements to the editing engine—performance optimizations, new language features, accessibility enhancements—flow downstream to Clover automatically. The editor is not a static component that Clover's developers must maintain in isolation; it is a living dependency that improves continuously. This is a deliberate engineering trade-off: rather than building a custom editor that would require ongoing investment to match the polish of established tools, Clover delegates the editing primitives to a best-in-class engine and focuses its own development effort on the integration layer—the connections between the editor and the AI assistant, the source manager, the compiler, and the version control system—where the real value of the application resides.

4.2 AI-Assisted Writing Through Claude Code

Adjacent to the editor panel, Clover provides an integrated terminal running Claude Code [11], Anthropic's agentic AI assistant. This is not a chatbot in a sidebar offering sentence-level suggestions, nor is it a cloud-hosted interface that requires the researcher to copy and paste text back and forth. It is a fully functional terminal session in which Claude Code operates directly on the project's files, reading the current state of every `.tex` file, the bibliography, the source materials registered in the source manager, and any configuration stored in the project's `CLAUDE.md` file. The AI shares the researcher's workspace in the most literal sense: it reads the same files, writes to the same files, and its output appears instantly in the editor panel.

The interaction model is conversational and iterative. A researcher might begin a writing session by asking Claude to draft an opening paragraph for a new section, providing only a brief description of the argument she wants to make. Claude reads the surrounding sections to understand the document's tone, structure, and level of technical detail, then produces a draft directly in the `.tex` file. The researcher reviews the result in the editor, decides that the second sentence needs to be stronger, and asks Claude to revise it with a more specific claim supported by a citation. Claude consults the bibliography, identifies the appropriate reference, tightens the sentence, and inserts a `\cite{}` command—all within seconds, all without the researcher touching her keyboard except to type the instruction.

This pattern scales to tasks of arbitrary complexity. A researcher can ask Claude to restructure an entire chapter, moving subsections into a more logical order and updating all internal cross-references. She can ask it to convert a set of informal notes into polished LaTeX prose, matching the style of the sections she has already written. She can describe a table's content in natural language and receive compilable LaTeX code that she can immediately preview. She can point Claude at a compilation error and ask it to diagnose and fix the problem, which Claude can do because it has access to the full project context, not just the isolated error message. She can ask Claude to review a paragraph for logical coherence, to suggest transitions between sections, or to identify places where the argument could be strengthened with additional evidence from the sources already registered in the project.

What distinguishes this from using a general-purpose AI assistant through a browser is the elimination of the integration barrier. When a researcher uses ChatGPT or Claude through a web interface, every interaction requires extracting a fragment of text from the editor, providing context that the model does not have, evaluating the response, and manually transferring the useful parts back into the document. This workflow is not merely tedious; it introduces systematic risks. Text generated outside the editor can introduce inconsistencies in notation, terminology, citation style, or formatting conventions that are difficult to catch during manual integration. In Clover, these risks vanish because the AI operates within the same project, reads the same files, and writes changes that are immediately subject to the same compilation and review pipeline as any other edit. The AI's output is not a suggestion to be integrated; it is an edit to be reviewed.

4.3 Git Diff View for Transparency

Every change that Claude Code makes to the manuscript—every new paragraph, every revised sentence, every inserted citation—appears as a modification to a plain text file tracked by Git. Clover's built-in diff view makes these changes visible in a format that researchers already understand: additions highlighted in green, deletions in red, unchanged context in white. This transparency is not an afterthought or a debugging convenience; it is a core design feature that addresses one of the deepest concerns surrounding AI-assisted writing.

When an AI writes or rewrites a passage, the researcher must be able to see exactly what changed. Without visibility into the specific modifications, she cannot evaluate whether the AI preserved her intended meaning, whether it introduced claims she did not make, whether it altered the emphasis of an argument, or whether it silently removed a qualification that she

considered important. The diff view provides this visibility in the most direct way possible: a line-by-line comparison between the previous state and the current state of the document. The researcher can accept the changes wholesale if they are satisfactory, revert specific lines or hunks if they are not, or use the diff as a starting point for her own further revisions. This workflow mirrors the code review practices that have become standard in software engineering, applied now to the domain of academic prose.

The combination of Git tracking and the diff view also creates an audit trail that has implications beyond the individual writing session. As academic publishers and funding agencies develop policies around AI-assisted writing, the ability to demonstrate exactly which portions of a manuscript were drafted, revised, or suggested by an AI agent becomes increasingly valuable. A Git history in which AI-generated changes are recorded as discrete, attributable commits provides documentation that no cloud-based "track changes" feature can match in granularity, portability, or verifiability. The researcher can show a journal editor or a peer reviewer precisely which commits involved AI assistance and which did not, with cryptographic guarantees that the history has not been tampered with. In an era when the provenance of academic text is under unprecedented scrutiny, this level of transparency is not a luxury but a professional necessity.

The diff view also serves a pedagogical function. By reviewing the changes that Claude makes, a researcher develops a more refined sense of how AI-assisted editing works—what the model tends to improve, where it tends to overreach, and what kinds of instructions produce the best results. Over time, this feedback loop makes the researcher a more effective collaborator with the AI, able to give more precise instructions and to catch potential issues more quickly. The transparency is bidirectional: it allows the human to evaluate the AI, and it allows the human to improve her own use of the AI.

4.4 PDF Compilation and Preview

Writing LaTeX without seeing the compiled output is like painting with your eyes closed. The gap between source code and rendered document is where formatting errors hide, where citation rendering goes wrong, where figure placement deviates from the author's intent, and where the visual rhythm of the page—margins, spacing, font choices, heading hierarchies—either supports or undermines the readability of the text. Closing this gap quickly and reliably is essential to a productive writing workflow.

In Clover, pressing Cmd+B on macOS or Ctrl+B on other platforms triggers a full LaTeX compilation using the locally installed TeX distribution. The compiled PDF appears in the preview panel within the same window, positioned alongside the editor so that the researcher can compare source and output without switching applications or managing separate windows. The compilation runs locally, on the researcher's own hardware, which means there is no server queue, no upload latency, no dependency on network speed, and no degradation during peak-usage periods. A researcher working on an airplane, in a library with spotty Wi-Fi, or at a conference hotel with overloaded internet can compile her document exactly as reliably as she can at her office desk.

The immediacy of this compile-review cycle transforms the way researchers interact with their documents. Rather than writing large blocks of LaTeX and compiling only periodically—a pattern that cloud-based editors with slow compilation pipelines inadvertently encourage—Clover's fast local compilation invites frequent, incremental compilation. The researcher writes a paragraph, compiles, checks the rendering, adjusts a spacing command, compiles again, and continues writing. This tight feedback loop catches formatting issues when they are small and recent, before they propagate through the document and become difficult to diagnose. It also reinforces the connection between the LaTeX source and the visual output, helping less experienced LaTeX users build intuition for how commands translate into rendered text.

The preview panel supports standard navigation: scrolling, zooming, and page jumping, allowing the researcher to inspect any part of the document without losing her place in the

editor. Because the compilation is triggered manually rather than running continuously on every keystroke, the researcher retains full control over when to compile, avoiding the distraction of a constantly updating preview during intensive editing sessions. This is a deliberate design choice that respects the writer's attention: the preview updates when asked, not when the system decides it should.

For projects that use BibTeX or Biber for bibliography management, the compilation pipeline handles the multiple passes required to resolve citations and cross-references. The researcher does not need to remember whether to run `pdflatex` once, twice, or three times, or when to interleave a `biber` invocation; the compilation command manages this sequence automatically. The result is a PDF that accurately reflects the current state of the manuscript, including all citations, cross-references, and bibliography entries, ready for review at a single keypress.

4.5 A Workspace, Not a Collection of Tools

The individual capabilities described in this chapter—a Monaco-powered editor, an AI writing assistant, a Git diff view, and a PDF compilation pipeline—are each valuable on their own. A researcher could, in principle, assemble an equivalent workflow from standalone applications: VS Code for editing, a terminal window for Claude Code, a Git client for change tracking, and a PDF viewer for compilation output. Many researchers do exactly this, and the resulting workflow, while functional, is burdened by the friction of constant context-switching, manual coordination, and the cognitive overhead of managing four or five windows that do not share state.

Clover's contribution is the integration. When Claude Code writes a paragraph, the researcher sees it appear in the editor panel without switching windows. When she compiles the document, the PDF preview updates in the same workspace without opening a separate application. When she reviews the AI's changes in the diff view, she can immediately revise the text in the editor and recompile to see the updated result. The panels are not merely co-located; they are connected through the shared project state—the filesystem, the Git repository, the bibliography, the source manager—so that actions in one panel have immediate, visible consequences in the others. This tight coupling between editing, AI assistance, change tracking, and compilation creates a workflow in which the mechanical overhead of academic writing—the switching, the copying, the waiting, the coordinating—is reduced to a minimum, leaving the researcher free to focus on what actually matters: the substance of her work.

By combining a professional-grade editor, an AI writing assistant that operates directly on project files, transparent and auditable change tracking through Git, and instant local compilation into a single, coherent interface, Clover removes the friction that has traditionally made LaTeX-based academic writing a test of patience as much as a test of scholarship. The editor is not merely a text input field with syntax highlighting; it is the center of a tightly integrated workspace in which every tool the researcher needs is immediately at hand, every AI contribution is visible and reviewable, and every compilation is one keypress away. The result is an environment that respects both the researcher's expertise and her time.

Chapter 5: Source Management: Your Research Hub

Research is built on sources—papers, datasets, notes, documentation, and the accumulated knowledge of a field distilled into citable artifacts. Managing these sources is one of the most time-consuming and cognitively demanding aspects of academic work. A researcher does not merely collect references; she must organize them, recall which ones are relevant to a given argument, extract the precise claims she needs, and ensure that every citation in her manuscript accurately reflects the source it purports to represent. Traditional reference managers like Zotero [3] and Mendeley [4] have long addressed the bibliographic dimension of this problem—storing metadata, generating formatted citations, syncing libraries across devices—but they operate as external tools, disconnected from the writing environment where citations actually matter. Clover's source management system takes a fundamentally different approach: it brings all of a researcher's reference materials directly into the writing environment, making them accessible not only to the researcher herself but also to the AI assistant that helps her write. The result is a system in which the gap between reading a source and citing it, between understanding a claim and incorporating it into a manuscript, collapses to almost nothing.

5.1 Adding and Registering Sources

Clover accepts a wide range of source types, reflecting the heterogeneous reality of modern research. Local files—PDFs of journal articles, CSV datasets, plain text notes, images of figures or whiteboard sketches—can be added directly from the researcher's filesystem. Web sources—academic papers hosted on arXiv or publisher websites, technical documentation, blog posts from research groups, official project pages—can be added by URL. Each source, regardless of its type or origin, is registered in a central JSON file called `sources.json` that serves as the project's knowledge base. This file is human-readable, version-controlled alongside the manuscript in Git, and structured so that both the researcher and the AI assistant can query it programmatically.

The registration process captures more than just a URL or file path. Each entry in `sources.json` includes a unique identifier, a descriptive name, the source type (paper, article, website, documentation, dataset), a free-text description summarizing the source's content and relevance, and—critically—a rich set of tags that encode the source's topical, methodological, and bibliographic metadata. The description field is particularly important because it provides the AI assistant with a natural-language summary of what each source contains, enabling it to decide which sources to consult for a given writing task without reading every source in full every time. A well-written description transforms `sources.json` from a mere index into a navigable map of the project's intellectual landscape.

The practical workflow for adding sources is designed to minimize friction. When a researcher discovers a relevant paper during a literature search, she can register it in Clover immediately, without leaving her writing environment. If the paper is a PDF on her desktop, she adds it as a local file. If she found it through Google Scholar or Semantic Scholar, she adds it by URL. If she wants to include her own notes on the paper—key claims, methodological concerns, passages she plans to cite—she can add those as a separate text source linked through tags to the original. Over the course of a writing project, `sources.json` grows into a comprehensive record of every piece of evidence the researcher has considered, accepted, or set aside, providing both a research audit trail and a living bibliography that evolves with the manuscript.

5.2 The Tag System: Structured Discovery

Every source in Clover is tagged with descriptive metadata drawn from multiple dimensions: domain (machine learning, biology, policy), method (survey, experiment, meta-analysis), concept (local-first, retrieval-augmented generation, citation management), author, venue, year, and more. These tags are not decorative; they are the primary mechanism by which Clover enables

discovery across a growing reference library. When a researcher is working on a section about a specific topic—say, the limitations of cloud-based writing tools—the tag system helps surface relevant sources she may not have immediately recalled, including sources she added weeks earlier for a different section but that happen to address the same concern from a different angle.

The tagging convention in Clover is deliberately granular. Each source carries a minimum of ten tags, and many carry substantially more. This density of metadata ensures that meaningful connections exist between sources even when their surface-level topics appear unrelated. A paper on CRDTs tagged with both "collaboration" and "offline-support" will surface when the researcher is writing about either topic, creating serendipitous connections that a keyword search on titles alone would miss. The tags are not hierarchical or drawn from a controlled vocabulary; they are free-form strings that the researcher defines according to her own conceptual framework, which means the tag system adapts to the researcher's way of thinking rather than imposing an external taxonomy.

This design reflects a deeper insight about how research knowledge is structured. Academic references do not exist in neat categories; they sit at the intersection of multiple themes, methods, and conversations. A single paper might be relevant to a discussion of automated scientific discovery, to a comparison of multi-agent architectures, and to an argument about the cost-effectiveness of AI-generated research. A flat, multi-tag system captures these overlapping relevancies in a way that folder hierarchies and single-category classifications cannot. When the AI assistant needs to find sources relevant to a paragraph it is drafting, it can query the tag system across multiple dimensions simultaneously, retrieving sources that match the intersection of the paragraph's topical concerns rather than relying on a single keyword match.

5.3 AI-Powered Source Utilization

The true value of Clover's source management becomes apparent when it intersects with the AI writing assistant. When Claude Code writes or edits content within Clover, it has access to every source registered in `sources.json`. It can read the descriptions and tags to identify which sources are relevant to the task at hand, consult the actual content of local files (PDFs, notes, datasets), and produce citations grounded in the researcher's specific reference material rather than in the model's general training data. This is a fundamentally different mode of AI-assisted writing from what a general-purpose chatbot provides.

Consider the difference in practice. When a researcher asks a general AI assistant to "write a paragraph about local-first software principles," the model draws on its training data—a vast but fixed corpus that may or may not include the specific papers the researcher considers authoritative. The resulting paragraph might be plausible but uncitable: it might paraphrase ideas from sources the researcher has never read, attribute claims to the wrong authors, or present a synthesis that does not match any single source closely enough to warrant a citation. The researcher must then verify every claim, track down the original sources, and manually insert citations—a process that can take longer than writing the paragraph from scratch.

In Clover, the same request produces a qualitatively different result. Claude Code reads `sources.json`, identifies the entries tagged with "local-first"—which might include Kleppmann et al.'s foundational paper [12], blog posts from the Ink & Switch research group, and the researcher's own notes on local-first design patterns—and drafts a paragraph that draws specifically on those sources, inserting `\cite{}` commands that reference the correct BibTeX keys from the project's `references.bib` file. The paragraph is not a hallucination dressed up with plausible-sounding citations; it is a synthesis of the researcher's own curated evidence, expressed in LaTeX and ready to compile. The AI's role shifts from generating content out of thin air to articulating the researcher's evidence in polished prose—a distinction that matters enormously for academic credibility.

This source-grounded approach also reduces the risk of inadvertent plagiarism and misattribution. Because Claude Code works from explicit source descriptions and tagged metadata rather

than from memorized training data, its outputs are traceable to specific entries in `sources.json`. The researcher can verify each citation against the actual source, confident that the AI's claims are derived from materials she has vetted rather than from an opaque training corpus. In an academic environment where the provenance of every claim is subject to scrutiny, this traceability is not merely convenient—it is essential.

5.4 Beyond NotebookLM: Sources as a Writing Substrate

Google's NotebookLM [5] pioneered the concept of source-grounded AI interaction for research. By requiring users to upload source documents before asking questions, NotebookLM demonstrated that constraining an AI's knowledge base to a curated set of references dramatically improves the reliability and relevance of its responses. The insight was powerful and well-timed: researchers who had grown frustrated with the hallucination tendencies of general-purpose chatbots found in NotebookLM a tool that could answer questions faithfully about their specific sources, summarize key findings, and even generate audio overviews of uploaded documents.

Clover takes this source-grounded philosophy and embeds it directly into the writing workflow, extending it from a read-and-query paradigm to a read-write-cite paradigm. In NotebookLM, the primary interaction is analytical: the researcher uploads sources, asks questions, and receives answers grounded in those sources. The output is insight—summaries, comparisons, extracted claims—that the researcher must then manually transfer into her manuscript, reformatting it for LaTeX, adding citations by hand, and ensuring consistency with the rest of the document. In Clover, the AI uses the same sources to write LaTeX directly, inserting properly formatted citations, matching the style and terminology of surrounding sections, and producing output that compiles without further intervention. The analytical step and the writing step are fused into a single operation.

Moreover, Clover's approach to source management offers capabilities that NotebookLM's architecture does not support. Clover's tag system enables multi-dimensional source discovery that goes beyond NotebookLM's document-level organization. The AI assistant in Clover can cross-reference sources across the entire project—not just within a single notebook—identifying connections between references that were added at different times for different purposes. Clover can also suggest additions to the source library: when the AI encounters a gap in the researcher's evidence—a claim that would benefit from a citation but for which no registered source provides support—it can recommend searching academic databases like Semantic Scholar [6] or SciSpace [8] for relevant literature, turning a passive reference library into an active research tool.

The local-first dimension adds a further distinction. NotebookLM is a cloud service: sources are uploaded to Google's servers, processed by Google's models, and stored in Google's infrastructure. For researchers working with sensitive data—unpublished manuscripts, proprietary datasets, materials under NDA, or simply work they prefer to keep private until publication—this cloud dependency is a significant concern. Clover's source management is entirely local. The `sources.json` file, the source documents themselves, and all AI processing occur on the researcher's own machine. No source material is transmitted to a remote server for indexing or storage. The researcher retains complete control over her intellectual property at every stage of the writing process, from source registration to final compilation.

5.5 A Living Knowledge Base

Source management in Clover is not a separate activity from writing—it is an integral part of the writing process itself. Adding a source is not a preliminary chore to be completed before writing begins; it is something that happens continuously, as the researcher discovers new evidence, refines her arguments, and responds to reviewer feedback. The tag system ensures that newly added sources are immediately discoverable by both the researcher and the AI, integrating seamlessly into the project's knowledge graph. The AI assistant ensures that sources are not merely

stored but actively used, transforming a passive reference library into a dynamic foundation for evidence-based writing.

This integration reflects a broader principle that animates Clover's design: the tools a researcher uses should not impose artificial boundaries between the phases of academic work. Reading sources, organizing references, drafting prose, inserting citations, and verifying claims are not discrete tasks to be performed in sequence with different applications; they are interleaved activities that flow into one another throughout the writing process. By unifying source management and AI-assisted writing within a single local-first environment, Clover ensures that every paragraph the researcher produces is grounded in real evidence, every citation points to a source she has actually read, and every claim is traceable to the materials that support it. The source manager is not a sidebar feature or an afterthought—it is the epistemic foundation on which the entire writing workflow rests.

Chapter 6: Clover Skills: Smart Writing Assistance

Beyond its editor and source manager, Clover ships with a set of built-in skills that automate common academic writing tasks. These are not generic AI prompts—the kind a researcher might type into a chat window and hope for the best. They are carefully designed workflows that guide the AI through complex, multi-step processes, encoding the tacit knowledge of experienced writers into repeatable, reliable procedures. Where a blank chat interface places the burden of prompting entirely on the user, Clover's skills structure the interaction so that the AI asks the right questions, consults the right sources, and produces output that is immediately useful within the LaTeX writing workflow. Each skill addresses a specific phase of the academic writing process—from initial ideation through literature discovery, source registration, and multilingual adaptation—and each is designed to reduce hours of manual labor to minutes of guided collaboration between the researcher and the AI.

The significance of this approach becomes clear when contrasted with how most researchers currently use AI assistants. A typical interaction with a general-purpose model begins with the researcher formulating a prompt, often spending considerable time figuring out how to describe what she needs in enough detail to get a useful response. She then evaluates the output, reformulates her request if the result was off-target, and repeats this cycle until the output is satisfactory—or until she gives up and does the work manually. This ad hoc prompting is inefficient not because the underlying model lacks capability but because the researcher lacks a structured framework for eliciting that capability. Clover's skills provide exactly this framework. They decompose complex tasks into well-defined steps, handle the prompting internally, and present the researcher with results that integrate directly into her project. The AI becomes not just a responder to questions but an active participant in a proven methodology.

6.1 Brainstorming: From Vague Ideas to Concrete Plans

Before a single word of LaTeX is written, the most critical phase of any writing project is the one that most tools ignore entirely: thinking. A researcher who begins drafting without a clear plan—without having articulated her purpose, identified her audience, considered the structure of her argument, or anticipated the constraints of her target venue—will inevitably produce a manuscript that wanders, that buries its contributions in unnecessary exposition, or that fails to address the concerns of its intended readers. The brainstorming skill exists to prevent this common and costly mistake by turning the nebulous early stage of a writing project into a structured ideation process.

When invoked, the brainstorming skill does not simply ask the researcher to "describe what you want to write." It initiates a guided conversation that covers the dimensions experienced academic writers consider instinctively but that less experienced writers often overlook. It asks about the paper's core purpose: is this a survey, a methods paper, an empirical study, a position piece? It asks about the intended audience: specialists in the field, a broader interdisciplinary readership, or a general scientific audience? It probes the structural constraints: what is the page limit, what sections does the venue require, what formatting conventions apply? It inquires about the competitive landscape: what existing papers address similar questions, and how does the researcher's contribution differ?

From the researcher's answers, the skill does not produce a single outline and call the job done. It proposes multiple approaches—different ways to frame the contribution, alternative structural organizations, various strategies for positioning the work within the existing literature—and presents the trade-offs of each. One approach might lead with the technical contribution and risk losing readers who lack the necessary background; another might begin with motivation and context, making the paper more accessible but potentially burying the novel content. The researcher evaluates these alternatives, selects elements from each, and the skill synthesizes the discussion into a design document: a structured blueprint that records the paper's purpose,

audience, key arguments, section outline, and critical decisions about scope and emphasis.

This design document is not a disposable artifact. It is saved within the project and becomes part of the context that Claude Code consults during subsequent writing sessions. When the AI later drafts a section, it can refer back to the design document to ensure that the prose aligns with the agreed-upon plan—that the introduction emphasizes the right motivations, that the related work section addresses the competitors identified during brainstorming, and that the conclusion returns to the themes established at the outset. The brainstorming skill thus serves a dual function: it helps the researcher clarify her thinking before writing begins, and it provides the AI with the persistent context it needs to produce coherent, on-target content throughout the writing process.

6.2 Paper Research: Assisted Literature Discovery

Literature review is simultaneously one of the most important and most tedious phases of academic work. A thorough review requires searching multiple databases, scanning hundreds of titles and abstracts, reading dozens of papers in detail, tracking citation chains forward and backward, and maintaining a mental model of how the discovered works relate to each other and to the researcher's own contribution. This process can consume days or weeks, and even diligent researchers routinely miss relevant work published in unfamiliar venues, in adjacent fields, or in languages they do not read. The paper-research skill transforms this labor-intensive process into an assisted, AI-guided workflow that dramatically accelerates discovery without sacrificing thoroughness.

The skill searches across multiple academic databases—including DBLP, Google Scholar, and arXiv—as well as the broader web, casting a wider net than any single database search could provide. When the researcher describes the topic she is investigating, the skill formulates targeted queries, retrieves candidate papers, and filters the results based on relevance to the researcher's specific project context. Because the skill has access to the project's existing `sources.json`, it can identify gaps in the current reference library and prioritize papers that address those gaps, rather than returning results the researcher has already seen.

A critical feature of the paper-research skill is its commitment to citation integrity. Every reference it discovers is verified through authoritative academic databases before being presented to the researcher. The skill never fabricates citations—a problem that has plagued general-purpose language models and eroded researcher trust in AI-generated bibliographies. If a paper cannot be verified through a reliable database, it is flagged rather than silently included. This verification step is essential because a single fabricated reference in a published manuscript can damage a researcher's credibility and undermine the trust that peer review is meant to establish.

Sources discovered through the paper-research skill are not merely listed in a chat response for the researcher to process manually. They are automatically added to the project's `sources.json` with proper metadata: a unique identifier, a descriptive name, the source type, a summary of the paper's content and relevance, tags that encode its topical and methodological attributes, and, when available, a formatted BibTeX citation entry. This automation means that the output of a literature search is immediately usable by both the researcher and the AI writing assistant. The newly discovered papers appear in the source manager, their tags integrate into the project's knowledge graph, and Claude Code can reference them in subsequent writing sessions without any manual registration step. What was previously an hours-long process of searching, evaluating, downloading, cataloging, and formatting references becomes a minutes-long interaction that yields a curated, project-ready bibliography.

6.3 Source Registration: Streamlined Reference Management

Not every source enters a project through a systematic literature search. Researchers discover relevant materials through serendipitous encounters—a colleague's recommendation, a paper

cited in a footnote, a blog post shared on social media, a technical report stumbled upon during a web search. The add-to-sources skill ensures that these ad hoc discoveries can be registered in the project's knowledge base with the same richness and consistency as sources found through the paper-research skill.

Whether the researcher is adding a local PDF downloaded to her desktop, a web article she found while browsing, or an academic paper she encountered on a publisher's website, the skill handles the entire registration process. It extracts metadata automatically where possible—parsing PDF headers for author names, titles, and publication details, scraping web pages for structured data, and consulting academic APIs for authoritative bibliographic information. It generates descriptive tags based on the source's content, ensuring that the new entry is immediately discoverable through the tag system described in Chapter 5. It formats the entry according to the conventions established in the project's `sources.json`, maintaining consistency across dozens or hundreds of registered sources.

The practical effect is a dramatic reduction in the friction of source management. Without this skill, registering a new source requires the researcher to manually create a JSON entry, fill in a unique identifier, write a description, compose a list of relevant tags, and ensure that the formatting is consistent with existing entries. This process is not technically difficult, but it is tedious enough that researchers frequently defer it—bookmarking papers "to add later" and then forgetting, or adding minimal entries that lack the rich metadata needed for effective AI-assisted writing. The add-to-sources skill eliminates this friction by handling the mechanical aspects of registration automatically, allowing the researcher to add a source in seconds and return immediately to writing. Every source added through the skill is immediately usable by Claude Code, ensuring that the project's knowledge base grows in lockstep with the researcher's evolving understanding of the literature.

6.4 Translation: Academic-Quality Multilingual Adaptation

Academic research is a global enterprise, and researchers increasingly work across linguistic boundaries. A Japanese researcher may need to draft a paper in English for an international conference. A French doctoral student may need to translate her advisor's working notes from French into English as the basis for a collaborative manuscript. A Chinese research group may want to prepare both an English submission and a Chinese-language summary for a domestic venue. In each of these scenarios, the translation must do more than convert words from one language to another—it must preserve technical terminology, maintain the precision of scientific claims, respect the conventions of academic register, and leave LaTeX formatting and citation structures intact.

The paper-translation skill provides exactly this capability. It performs academic-quality translation that is aware of the document's technical context, preserving domain-specific terminology that a general-purpose translator might paraphrase or mistranslate. Mathematical notation, LaTeX commands, `\cite{}` references, cross-reference labels, and environment structures pass through the translation unchanged, so that the translated document compiles without modification. The skill understands that academic prose has conventions that differ from conversational language—the passive constructions common in scientific English, the hedging language used to qualify empirical claims, the precise vocabulary that distinguishes "significant" in its statistical sense from "significant" in its colloquial sense—and it preserves these conventions in the target language.

This is particularly valuable for researchers in multilingual academic communities, where the same work may need to be presented in different languages for different audiences. Rather than maintaining two separate manuscripts that inevitably drift apart as revisions accumulate, a researcher can write in her strongest language and use the translation skill to produce a high-quality adaptation in the target language, updating the translation as the source manuscript evolves. The result is not a rough draft that requires extensive manual polishing; it is a publication-ready

translation that preserves the structure, the citations, and the technical precision of the original. For non-native English speakers who constitute the majority of the global research community, this capability can mean the difference between a manuscript that reads fluently and one that is marked—however unfairly—by the linguistic traces of translation.

6.5 The Philosophy Behind Skills

Clover's built-in skills represent a deliberate philosophy of AI assistance that stands in contrast to the dominant paradigm of general-purpose chat interfaces. The prevailing approach in most AI writing tools is to provide a blank text field and invite the user to type whatever comes to mind. This design maximizes flexibility but minimizes guidance: the researcher must already know what to ask, how to ask it, and how to evaluate whether the response is adequate. For experienced AI users who have developed effective prompting strategies through trial and error, this flexibility is sufficient. For everyone else—which is to say, for most researchers—it is a recipe for frustration, wasted time, and suboptimal results.

Clover's skills take the opposite approach. Rather than leaving the researcher to discover effective workflows on her own, each skill encodes a proven methodology for a specific task. The brainstorming skill encodes the questions that experienced writers ask before they begin drafting. The paper-research skill encodes the search strategies, verification procedures, and metadata conventions that thorough literature reviews require. The add-to-sources skill encodes the registration workflow that keeps a project's knowledge base consistent and complete. The translation skill encodes the linguistic and technical awareness that academic translation demands. In each case, the AI does not merely answer questions—it guides the researcher through a structured process that produces reliable, high-quality results.

This skill-based architecture also means that Clover's capabilities grow over time as new skills are developed and refined. Each skill is a self-contained module that can be improved independently, and new skills can be added to address writing tasks that the current set does not cover—manuscript formatting for specific journals, automated consistency checking across sections, generation of reviewer response letters, or structured peer feedback. The skill system is not a fixed menu of features but an extensible framework that encodes and distributes expert knowledge, making the collective wisdom of experienced academic writers available to every researcher who uses Clover. The AI does not replace the researcher's judgment; it amplifies her capabilities by ensuring that every task she undertakes is supported by a workflow designed to produce the best possible outcome.

Chapter 7: Getting Started & Future Directions

Getting started with Clover requires minimal setup and no proprietary accounts, institutional licenses, or cloud subscriptions. Unlike platforms that demand onboarding through a web interface controlled by the provider, Clover asks only that the researcher install three freely available components on her own machine—and then hands her the keys. This chapter walks through installation, project creation, and the basic writing workflow, before looking ahead to the features and directions that will shape the application's future. The goal is to leave the reader equipped: ready to launch Clover and begin writing her next paper in an environment where the AI works for her rather than the other way around.

7.1 Prerequisites

Clover's dependency footprint is deliberately small. The application requires exactly three components, each of which is open, well-documented, and available at no cost.

The first is Node.js, version 18 or later, which provides the JavaScript runtime powering Clover's Electron-based desktop application and its Express-based web server. Researchers who have worked with modern web development or data-science tooling will likely have Node.js installed already; those who do not can obtain it from the official website or through a version manager such as `nvm`. Installation is straightforward on macOS, Windows, and Linux.

The second prerequisite is a TEX distribution for local LATEX compilation: TeX Live for Linux, MacTeX for macOS, or MiKTeX for Windows. Each ships with the standard packages that the overwhelming majority of academic documents require and provides a package manager for installing additional ones on demand. Researchers who already write in LATEX—the primary audience for Clover—will almost certainly have a distribution installed already.

The third component is Claude Code [11], Anthropic's terminal-based AI assistant, which powers Clover's drafting, editing, source-grounded analysis, brainstorming, and literature research capabilities. Installing Claude Code requires an Anthropic account and is accomplished through a single command-line invocation. API usage incurs costs based on token consumption, but the tooling itself is freely available and extensively documented.

It is worth emphasizing what is *not* required. There is no Clover account, no subscription tier, no institutional license, and no cloud service that must be running for the application to function. The three prerequisites are tools the researcher installs on her own machine and controls entirely—a direct consequence of the local-first philosophy described in Chapter 2.

7.2 Creating a Project

With the prerequisites in place, launching Clover and creating a new project is a matter of seconds rather than minutes. The researcher clones the Clover repository from GitHub[1], installs the application's dependencies with a single `npm install` command, and starts the application. Clover opens to a clean workspace where the researcher can create a new project by selecting a directory on her local filesystem. The application generates a lightweight configuration file within the chosen directory and prepares the workspace for writing.

The project directory is immediately a standard filesystem directory containing plain `.tex` files, a `.bib` bibliography, and a `.clover` configuration folder that stores the source registry and project settings. There is no database, no binary format, and no proprietary structure. The researcher can inspect every file with a text editor, track every change with Git, and back up the entire project by copying a folder. If she already has a LATEX manuscript in progress, she can point Clover at the existing directory and begin working immediately—the application adapts to the project rather than demanding that the project conform to the application.

[1] https://github.com/yoichi1484/clover

Once the project is created, the researcher populates her knowledge base by adding sources through the source management panel described in Chapter 5—PDFs of key papers, URLs of relevant articles, or notes from preliminary research. Each source is registered with metadata that makes it discoverable by both the researcher and the AI assistant: a descriptive name, topical tags, and, for academic papers, a BibTeX citation entry. The richer the knowledge base, the more effectively Claude Code can ground its writing assistance in the researcher's actual evidence rather than in the model's general training data.

7.3 The Basic Writing Workflow

A typical Clover session follows a natural rhythm that mirrors the way experienced academic writers already think about their work, with the critical difference that AI assistance is woven into every step rather than bolted on as an afterthought.

The session often begins with a review of sources. The researcher opens the source management panel, scans registered materials, and identifies the papers most relevant to the section she plans to write. She may add new sources, tag existing ones, or use the paper-research skill to search for literature she may have missed—ensuring the AI has access to current evidence when writing begins.

With sources assembled, the researcher engages Claude Code in the terminal panel. She describes the section she wants to draft, specifying the argument, the evidence, and the tone. Claude Code, which has access to the entire project—every `.tex` file, every registered source, the `CLAUDE.md` configuration—produces a draft directly in the appropriate file. The draft appears in the editor in real time, syntax-highlighted and immediately editable. The researcher reviews the output, makes adjustments, asks Claude to revise specific passages, and iterates until the text meets her standards.

When a section reaches a satisfactory state, the researcher compiles with Cmd+B (or Ctrl+B on Windows and Linux). The PDF appears in the preview panel within seconds. She checks formatting, verifies citations, and confirms figure placement. If compilation produces errors, Claude Code can diagnose and fix them with full project context.

With the output reviewed, the researcher commits her changes using Git. Clover's diff view shows exactly what was added or modified, providing a final review opportunity before changes are recorded in the project's history. Each cycle of source review, AI-assisted drafting, compilation, and commitment builds on the last, with the knowledge base and version history growing richer with every session.

This workflow is not a rigid sequence. Some sessions are entirely manual; others are heavily AI-driven. Some focus on compilation and formatting rather than prose. The rhythm adapts to the researcher's needs on any given day, and the integrated environment ensures that whichever mode she chooses, every tool is immediately at hand without the context-switching that fragmented workflows impose.

7.4 Future Directions

Clover is under active development, and its current capabilities—while already sufficient for productive academic writing—represent only the beginning of what an integrated, local-first, AI-powered writing environment can become. The roadmap reflects a commitment to deepening every dimension of the application without compromising the architectural principles that define it.

The most immediate frontier is deeper AI integration. As large language models continue to improve in reasoning, factual accuracy, and domain expertise, Clover will expose these capabilities through new skills and more sophisticated interaction patterns. Future versions may support AI-assisted outlining that generates not just section headings but detailed argumentative structures, AI-driven consistency checking that identifies contradictions between different parts

of a manuscript, and AI-powered formatting assistance that automatically adapts a document to the submission requirements of a specific journal or conference. The Model Context Protocol (MCP) supported by Claude Code [11] provides the extensibility mechanism for these enhancements: new capabilities can be added as MCP tools without modifying the core application, and researchers can develop custom integrations tailored to the needs of their specific disciplines.

Expanded source type support is another priority. The current source manager handles PDFs, web pages, and text-based materials effectively, but academic research increasingly involves datasets, code repositories, multimedia content, experimental logs, and structured data in formats ranging from CSV to JSON to domain-specific schemas. Future versions of Clover will broaden the source manager to ingest and index these diverse materials, allowing the AI to ground its writing assistance not only in published literature but in the researcher's own experimental data, computational notebooks, and raw observations. The vision is a knowledge base that captures the full breadth of a research project's evidentiary foundation, not merely its bibliographic references.

Collaboration features represent a particularly interesting design challenge for a local-first application. The goal is to enable multi-author workflows—shared editing, review annotations, coordinated revision cycles—without abandoning the principle that the researcher's data remains on her own machine. Git already provides the foundation for asynchronous collaboration through branching, merging, and pull requests, and Clover will build on this foundation with features that make Git-based collaboration more accessible to researchers who are not experienced software developers. Simplified conflict resolution interfaces, integrated commenting systems that store annotations as version-controlled files rather than platform-locked metadata, and workflow templates for common multi-author scenarios are all under consideration. The guiding constraint is that collaboration should enhance the local-first experience rather than undermining it: every collaborator retains a full copy of the project, every contribution is attributable and auditable, and no central server is required for the collaborative process to function.

Community-contributed skills offer a path toward capabilities that no single development team could anticipate alone. The skill architecture described in Chapter 6 is designed to be extensible, and future versions of Clover will provide a framework for researchers to create, share, and install custom skills encoding domain-specific workflows—a bioinformatics skill for gene-expression tables, a social-science skill for qualitative-research reporting standards, a mathematics skill for proof formalization in LaTeX. By opening the skill system to community contributions, Clover can evolve from a general-purpose writing tool into a platform carrying the collective methodological expertise of diverse research communities.

Finally, tighter integration with academic publishing workflows will reduce the friction between writing and submitting. Future versions may include submission-preparation skills that automatically reformat a manuscript for a specific venue, generate the required cover letter, compile supplementary materials, and verify that the submission package meets technical requirements. Integration with preprint servers such as arXiv could streamline posting drafts and managing publicly available versions.

These future directions share a common thread: each deepens Clover's capabilities while preserving the local-first, researcher-controlled architecture that distinguishes it from cloud-dependent alternatives. The application grows more powerful not by accumulating platform dependencies but by enriching the local workspace with new skills, source types, collaboration mechanisms, and publishing integrations—all operating on the same principles of plain files, standard formats, and transparent AI assistance that define Clover today.

7.5 Conclusion

Clover is more than a tool—it is a statement about how academic writing should work in the age of artificial intelligence. The prevailing trajectory in the industry points toward ever-greater centralization: cloud platforms that host your manuscript on their servers, AI assistants that

require your data to pass through their infrastructure, collaboration systems that lock your workflow into a single provider's ecosystem. This trajectory is understandable from a business perspective, but it is corrosive from a scholarly one. Researchers who depend on centralized platforms surrender control over their most valuable intellectual output to organizations whose incentives may not align with the long-term interests of scholarship.

Clover charts a different course. By keeping the researcher in control—of her files, her tools, her data, and her choice of AI assistant—it demonstrates that a writing environment can be simultaneously powerful and respectful of the people who use it. The AI's contributions appear as transparent, reviewable edits in a Git-tracked workspace. The source manager indexes local files that never leave the researcher's machine. The compiler runs on local hardware with predictable performance, free from server queues and outages.

What makes Clover valuable is not any single feature in isolation—LaTeX editors, reference managers, AI assistants, and version control systems all exist independently—but their unification within an environment designed from the ground up for academic writing. The distance between reading a source and citing it, between asking an AI for help and seeing the result in your manuscript, between compiling and reviewing the output, collapses to zero. The researcher's cognitive effort goes where it belongs: into thinking, arguing, and writing, rather than into managing the logistics of a fragmented toolchain.

Clover is open source, available on GitHub, and welcoming of contributions from the research community it seeks to serve. It points toward a future where AI amplifies human expertise rather than replacing it, where tools respect the autonomy of the people who depend on them, and where the act of writing a research paper is supported by technology that is as transparent, portable, and enduring as the scholarship it helps produce. The first step is to install it, create a project, and start writing. The rest follows from there.

References

[1] OpenAI. *Prism: AI-Powered Scientific Writing Workspace*. https://openai.com/prism/. 2026.

[2] Overleaf. *Overleaf: Online LaTeX Editor*. https://www.overleaf.com/. 2024.

[3] Corporation for Digital Scholarship. *Zotero: Your Personal Research Assistant*. https://www.zotero.org/. Open-source reference manager with browser integration and 9,000+ citation styles. 2024.

[4] Elsevier. *Mendeley: Free Reference Manager*. https://www.mendeley.com/. Cloud-based reference manager with PDF annotation and Scopus integration. 2024.

[5] Google. *NotebookLM: AI-Powered Research Assistant with Deep Research*. https://notebooklm.google.com/. Source-grounded AI assistant with Deep Research and Audio Overview features. 2025.

[6] Elicit. *Elicit: AI for Scientific Research*. https://elicit.com/. 2025.

[7] Typst GmbH. *Typst: The New Foundation for Documents*. https://typst.app/. Modern typesetting system written in Rust with fast compilation and Markdown-inspired syntax. 2025.

[8] SciSpace. *SciSpace AI Research Agent*. https://scispace.com/. 2025.

[9] Chris Lu et al. "The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery". In: *arXiv preprint arXiv:2408.06292* (2024).

[10] Samuel Schmidgall et al. "Agent Laboratory: Using LLM Agents as Research Assistants". In: *arXiv preprint arXiv:2501.04227* (2025).

[11] Anthropic. *Claude Code: AI Pair Programming Assistant*. https://claude.com/code. Terminal-based AI coding assistant with multi-agent support and MCP integration. 2025.

[12] Martin Kleppmann et al. "Local-first software: you own your data, in spite of the cloud". In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)* 2019, pp. 154–178. DOI: 10.1145/3359591.3359737.